

```
--- djangoapi\core\myLib\geometryTools.py ---
```

```
from django.db import connection
```

```
from djangoapi.settings import EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION
```

```
class WkbConversor:
```

```
    def __init__(self, epsg_for_geometries: str=EPSG_FOR_GEOMETRIES,  
                  st_snap_precision: float=ST_SNAP_PRECISION,  
                  snap_to_grid: bool = True):
```

```
        self.epsg_for_geometries=epsg_for_geometries
```

```
        self.st_snap_precision=st_snap_precision
```

```
        self.snap_to_grid=snap_to_grid
```

```
        self.__wkb=None
```

```
    def set_wkt_from_text(self, geom_text:str)-> str:
```

```
        """
```

```
        Receives a string, with a geojson, or wkt geometry,  
        and return it in wkt.
```

```
        If the self.st_snap_precision is true the vertices are rounded
```

```
        """
```

```
        if 'coordinates' in geom_text:
```

```
            print('geojson')
```

```
            return self.__set_wkb_from_geojson(geom_text)
```

```
        else:
```

```
            print('wkt')
```

```
            return self.__set_wkb_from_wkt(geom_text)
```

```
    def set_wkb_from_wkb(self, wkb):
```

```
        self.__wkb=wkb
```

```
    def __set_wkb_from_geojson(self, geojson:str)->str:
```

```
        cursor = connection.cursor()
```

```
        #Ejecuta la función PostGIS ST_GeomFromText para convertir WKT a WKB
```

```
        print('set_wkb_from_geojson')
```

```
        if self.snap_to_grid:
```

```
            q="""SELECT  
                ST_SNAPTOGRID(  
                    st_setsrid(  
                        ST_GeomFromGeoJSON(%s)  
                        ,%s  
                    ),  
                    %s  
                )  
            """
```

```
        else:
```

```
            q="""SELECT  
                st_setsrid(  
                    ST_GeomFromGeoJSON(%s)  
                    ,%s  
                ),  
                %s  
            """
```

```

        """
        cursor.execute(q, [geojson, self.epsg_for_geometries, self.st_snap_precision])
        row = cursor.fetchone()
        self.__wkb=row[0]
        return self.get_as_wkb() # Esto será el WKB

def __set_wkb_from_wkt(self, wkt:str):
    cursor = connection.cursor()
    #Ejecuta la función PostGIS ST_GeomFromText para convertir WKT a WKB
    print('set_wkb_from_wkt')
    if self.snap_to_grid:
        q="""SELECT
            ST_SNAPTOGRID(
                st_setsrid(
                    ST_GeomFromText(%s)
                    ,%s
                ),
                %s
            )
        """
    else:
        q="""SELECT
            st_setsrid(
                ST_GeomFromText(%s)
                ,%s
            ),
            %s
        """
    cursor.execute(q, [wkt, self.epsg_for_geometries, self.st_snap_precision])
    row = cursor.fetchone()
    self.__wkb=row[0]
    return self.get_as_wkb()

    def set_wkb_from_table(self, table_name:str, id_to_select:int,
geom_field_name:str='geom')->str:
    cursor = connection.cursor()
    #Ejecuta la función PostGIS ST_GeomFromText para convertir WKT a WKB
    print('set_wkb_from_table')
    if self.snap_to_grid:
        q=f"""SELECT ST_SNAPTOGRID({geom_field_name})
            FROM {table_name} WHERE id = %s
        """
    else:
        q=f"""SELECT {geom_field_name}
            FROM {table_name} WHERE id = %s
        """
    cursor.execute(q, [id_to_select])
    l = cursor.fetchall()
    if len(l)==0:
        raise Exception(f"No record with the id {id_to_select} in the table {table_name}")

    self.__wkb=l[0][0]

```

```

        return self.get_as_wkb()

def get_as_wkb(self):
    """
    Returns the snapped geometry in wkb
    """
    return self.__wkb

def get_as_geojson(self):
    """
    Returns the snapped geometry in geojson
    """
    query="SELECT ST_AsGeojson(%s)"
    cursor=connection.cursor()
    cursor.execute(query, [self.get_as_wkb()])
    row = cursor.fetchone()
    return row[0] if row else None #Devuelve la geometría en formato geojson o None

def get_as_wkt(self):
    """
    Returns the snapped geometry in wkt
    """
    query="SELECT ST_AsText(%s)"
    cursor=connection.cursor()
    cursor.execute(query, [self.get_as_wkb()])
    row = cursor.fetchone()
    return row[0] if row else None #Devuelve la geometría en formato geojson o None

class GeometryChecks:
    def __init__(self, wkb: str):
        self.wkb=wkb
        self.related_ids = None
        self.requested_relation = None
        self.table_name = None

class GeometryChecks:
    def __init__(self, wkb: str):
        self.wkb=wkb
        self.related_ids = None

    def is_geometry_valid(self):
        """Checks if a geometry in geojson is valid."""
        print('is_geometry_valid')
        cursor=connection.cursor()
        q="SELECT ST_IsValid(%s)"
        cursor.execute(q, [self.wkb])
        row = cursor.fetchone()
        #row is true or false
        return row[0]

    def check_st_relate(self, table_name: str, matrix9IM: str,
id_to_avoid:int=None)->list:
        """Checks whether or not exists a geometry with the relation of the geom with all

```

```

the geometries in the layer
    layername using the matrix 9IM. The geom is in geojson format.
    """

    cursor=connection.cursor()
    if id_to_avoid is None:
        q=f""""SELECT id FROM {table_name} WHERE ST_relate(geom,%s,%s)""""
        values=[self.wkb, matrix9IM]
    else:
        q=f""""SELECT id FROM {table_name} WHERE ST_relate(geom,%s,%s) and id !=
%s""""
        values=[self.wkb, matrix9IM, id_to_avoid]
    cursor.execute(q, values)
    self.related_ids=cursor.fetchall()
    self.requested_relation= f'ST_relate, matrix: {matrix9IM}'
    self.table_name=table_name
    return self.related_ids # Devuelve los ids de las geometrías que cumplen la
relación

    def check_st_condition(self, table_name: str, st_condition: str,
id_to_avoid:int=None)->list:
    """Checks whether or not exists a geometry with the condition
    st_condition: st_intersects, st_contains, st_within, ...
    of the current geom with all the geometries in the layer
    layername.
    """
    cursor=connection.cursor()
    if id_to_avoid is None:
        q=f""""SELECT id FROM {table_name} WHERE {st_condition}(geom,%s)""""
        values=[self.wkb]
    else:
        q=f""""SELECT id FROM {table_name} WHERE {st_condition}(geom,%s) and id !=
%s""""
        values=[self.wkb, id_to_avoid]

    cursor.execute(q, values)
    self.related_ids=cursor.fetchall()
    self.requested_relation= st_condition
    self.table_name=table_name
    return self.related_ids

def are_there_related_ids(self)->bool:
    """
    First must call check_st_relate, or raise exception
    If there are related ids returns true
    """
    if self.related_ids is None:
        raise Exception("You first have to call a check_method")
    else:
        if len(self.related_ids)==0:
            return False
        else:
            return True

```

```
def get_relate_message(self)->str:
    if self.are_there_related_ids():
        return f"The following ids of the table {self.table_name} have the requested
relation ({self.requested_relation}), with the given geometry: {self.related_ids}"
    else:
        return "There are not geometries with the requested relation"
```